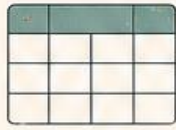




¿QUÉ ES UNA VISTA?



Una vista es una tabla virtual basada en el resultado de una consulta. No almacena datos físicamente, sino que muestra los datos de una o más tablas cada vez que se consulta.



¿PARA QUÉ SIRVE?

- ✓ Simplifica consultas complejas.
- ✓ Mejora la seguridad (ocultando columnas sensibles).
- ✓ Presenta datos de forma personalizada.
- ✓ Facilita la reutilización de consultas.
- ✓ Puede unificar datos de varias tablas.

CARACTERÍSTICAS

-  No almacena datos.
-  Los datos siempre están actualizados.
-  Permite controlar el acceso a la información.
-  Puede basarse en una o más tablas.

1. SINTAXIS: CREAR VISTA

```
CREATE VIEW nombre_vista
AS
SELECT columna1, columna2, ...
FROM tabla1
[JOIN tabla2 ON condición]
[WHERE condición]
[GROUP BY columna]
[HAVING condición]
[ORDER BY columna];
```

- **CREATE VIEW:** crea la vista.
- **nombre_vista:** nombre que tendrá la vista.
- **AS:** indica que lo que sigue es la consulta que define la vista.
- La consulta (**SELECT**): define los datos que mostrará la vista.

EJEMPLO

```
-- Vista de estudiantes con su facultad
CREATE VIEW vw_Estudiantes_Facultad
AS
SELECT e.IdEstudiante,
       e.Nombres,
       e.Apellidos,
       f.NombreFacultad
FROM   Academico.Estudiante e
       INNER JOIN Academico.Facultad f
           ON e.IdFacultad = f.IdFacultad
WHERE  e.Activo = 1;
```

2. SINTAXIS: ALTERAR Y ELIMINAR VISTA

ALTERAR VISTA

```
ALTER VIEW nombre_vista
AS
-- nueva consulta
SELECT ...
FROM ...;
```



ELIMINAR VISTA

```
DROP VIEW nombre_vista;
```



USO:

```
SELECT *
FROM vw_Estudiantes_Facultad;
```



3. TIPOS DE VISTAS



Vista simple

Basada en una sola tabla.
Generalmente es actualizable.



Vista compleja

Basada en múltiples tablas, con joins, agrupaciones, etc.
Generalmente NO es actualizable.

4. ACTUALIZABILIDAD DE VISTAS



Actualizable

Permite INSERT, UPDATE, DELETE si cumple estas reglas:

- Basada en una sola tabla.
- No usa GROUP BY, DISTINCT, TOP, etc.
- Todas las columnas provienen directamente de la tabla base.



No actualizable

No permite modificaciones directas cuando usa:

- JOIN con múltiples tablas.
- Funciones de agregación.
- DISTINCT, GROUP BY, UNION, etc.

5. BUENAS PRÁCTICAS

- ✓ Usa nombres descriptivos (vw_ o prefijos institucionales).
- ✓ Documenta el propósito de la vista.
- ✓ Evita usar SELECT * en la vista.
- ✓ Utiliza vistas para seguridad y simplificación.
- ✓ Revisa el rendimiento de consultas basadas en vistas.

EJEMPLO PRÁCTICO

Objetivo: Mostrar los estudiantes activos de la Facultad de Ingeniería.

```
CREATE VIEW vw_Estudiantes_Ingenieria AS
SELECT e.IdEstudiante, e.Nombres, e.Apellidos, e.Codigo,
       p.NombrePrograma
FROM   Academico.Estudiante e
       INNER JOIN Academico.ProgramaEstudios p
           ON e.IdPrograma = p.IdPrograma
WHERE  p.IdFacultad = (SELECT IdFacultad FROM Academico.Facultad
                       WHERE NombreFacultad = 'Ingenieria')
AND    e.Activo = 1;
```

USO:

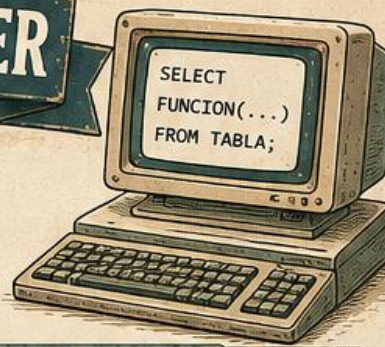
```
SELECT *
FROM vw_Estudiantes_Ingenieria
ORDER BY Apellidos;
```

RECUERDA

Las vistas son herramientas poderosas para organizar, proteger y simplificar el acceso a la información.

¡Úsalas sabiamente!

Las funciones son procedimientos predefinidos que realizan cálculos, transforman datos y devuelven un valor. Facilitan la escritura de consultas y mejoran su legibilidad.



¿QUÉ ES UNA FUNCIÓN?



Es un objeto que recibe uno o más valores (argumentos), realiza una operación y devuelve un único valor.

Argumentos → **FUNCIÓN()** → Valor Devuelto

¿PARA QUÉ SIRVEN?

- ✓ Realizar cálculos y operaciones.
- ✓ Convertir y dar formato a datos.
- ✓ Obtener información del sistema.
- ✓ Simplificar consultas complejas.
- ✓ Mejorar la legibilidad del código.



SINTAXIS GENERAL

```
SELECT FUNCION (argumento1, argumento2, ...)
FROM tabla;
```

FUNCION : nombre de la función predefinida.

argumento : puede ser una columna, expresión o valor literal.

Devuelve : un único valor por cada fila procesada.

CATEGORÍAS DE FUNCIONES



1. FUNCIONES DE CADENA

Manipulan valores de texto.

- LEN()
- UPPER()
- LOWER()
- SUBSTRING()
- CONCAT()
- LTRIM() / RTRIM()

Ejemplo:

```
SELECT UPPER(Nombres) AS NomMayus
FROM Estudiante;
```



2. FUNCIONES NUMÉRICAS

Realizan cálculos matemáticos.

- ABS()
- ROUND()
- CEILING()
- FLOOR()
- POWER()
- SQRT()

Ejemplo:

```
SELECT ROUND(Promedio, 2) AS Prom
FROM Notas;
```



3. FUNCIONES DE FECHA Y HORA

Trabajan con fechas y horas.

- GETDATE()
- DATEADD()
- DATEDIFF()
- CONVERT()
- YEAR() / MONTH() / DAY()

Ejemplo:

```
SELECT DATEDIFF(DAY, FechaInicio,
GETDATE()) AS DiasTranscurridos
FROM Tramite;
```



4. FUNCIONES LÓGICAS

Devuelven valores lógicos.

- CASE
- ISNULL()
- NULLIF()
- COALESCE()

Ejemplo:

```
SELECT ISNULL(Correo, 'Sin correo')
AS CorreoElectronico
FROM Persona;
```



5. FUNCIONES DE AGREGACIÓN

Resúmen datos de un conjunto.

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

Ejemplo:

```
SELECT COUNT(*) AS TotalAlumnos
FROM Estudiante;
```

EJEMPLOS PRÁCTICOS

FUNCIÓN	SINTAXIS	DESCRIPCIÓN	EJEMPLO
LEN()	LEN(cadena)	Devuelve la cantidad de caracteres de una cadena.	SELECT LEN('Universidad UPLA') AS Longitud;
UPPER()	UPPER(cadena)	Convierte a mayúsculas.	SELECT UPPER('upla') AS Mayusculas;
SUBSTRING()	SUBSTRING(cadena, inicio, longitud)	Extrae una parte de la cadena.	SELECT SUBSTRING('Ingeniería de Sistemas', 1, 10) AS Subtexto;
GETDATE()	GETDATE()	Devuelve la fecha y hora actual del servidor.	SELECT GETDATE() AS FechaHoraActual;
DATEADD()	DATEADD(intervalo, número, fecha)	Suma o resta un intervalo de tiempo a una fecha.	SELECT DATEADD(DAY, 7, GETDATE()) AS FechaMas7Dias;
DATEDIFF()	DATEDIFF(intervalo, fecha1, fecha2)	Devuelve la diferencia entre dos fechas.	SELECT DATEDIFF(MONTH, '2024-01-01', GETDATE()) AS MesesTranscurridos;
ISNULL()	ISNULL(expresión, reemplazo)	Reemplaza NULL por un valor especificado.	SELECT ISNULL(Telefono, 'No registrado') AS Telefono FROM Persona;
CASE	CASE WHEN condición THEN valor1 [ELSE valor2] END	Evalúa condiciones y devuelve resultados.	SELECT CASE WHEN Nota >= 13 THEN 'Aprobado' ELSE 'Desaprobado' END AS Resultado FROM Notas;

ORDEN DE EVALUACIÓN

- 1 Se evalúan los argumentos.
- 2 La función procesa los valores.
- 3 Devuelve un único resultado.



CONSEJOS

- ★ Usa funciones para simplificar y hacer más legibles tus consultas.
- ★ Combina funciones para obtener resultados más precisos.
- ★ Revisa el tipo de dato devuelto por cada función.



NOTA IMPORTANTE

Algunas funciones pueden afectar el rendimiento de las consultas si se usan en columnas indexadas dentro de cláusulas WHERE. Úsalas con criterio.



LAS FUNCIONES SON HERRAMIENTAS PODEROSAS PARA TRANSFORMAR DATOS Y OBTENER INFORMACIÓN ÚTIL DE MANERA EFICIENTE.



SCHEMAS (ESQUEMAS)

EN SQL SERVER



Los esquemas organizan y agrupan objetos de la base de datos para mejorar la seguridad, el orden y la administración.

¿QUÉ ES UN ESQUEMA?

Un esquema es un contenedor lógico dentro de una base de datos que puede contener objetos como tablas, vistas, procedimientos almacenados, funciones, etc.



Permite agrupar objetos relacionados por área o función.

¿PARA QUÉ SIRVE?

- ✓ Organiza los objetos de la base de datos.
- ✓ Mejora la seguridad y el control de acceso.
- ✓ Evita conflictos de nombres de objetos.
- ✓ Facilita el mantenimiento y la administración.
- ✓ Permite trabajar con múltiples equipos sin interferencias.

SINTAXIS BÁSICA

```
-- Crear esquema
CREATE SCHEMA nombre_esquema;

-- Eliminar esquema
DROP SCHEMA nombre_esquema;

-- Ver esquemas existentes
SELECT name
FROM sys.schemas
ORDER BY name;
```

ESTRUCTURA ORGANIZADA CON ESQUEMAS



EJEMPLO DE USO

En la base de datos "GradosTitulos", se utilizan esquemas para separar por módulos:

ESQUEMA	PROPÓSITO
Catalogo	Tablas de parámetros y catálogos.
Academico	Entidades académicas principales.
Tramite	Proceso administrativo de trámites.
Evaluacion	Evaluaciones y sustentaciones.
Documento	Resoluciones y documentos oficiales.
Seguridad	Usuarios, roles y permisos.

CREAR ESQUEMAS EN SQL SERVER

```
USE GradosTitulos;
GO
-- Crear esquemas
CREATE SCHEMA Catalogo;
CREATE SCHEMA Academico;
CREATE SCHEMA Tramite;
CREATE SCHEMA Evaluacion;
CREATE SCHEMA Documento;
CREATE SCHEMA Seguridad;
```

¡IMPORTANTE!

Cada objeto debe pertenecer a un esquema.

Ejemplo:

```
CREATE TABLE Academico.Estudiante (
  IdEstudiante INT PRIMARY KEY,
  Nombres VARCHAR(100)
);
```

BUENA PRÁCTICA

Usa nombres de esquemas claros y representativos del área o módulo.

- ✓ Evita el esquema dbo para objetos de usuario.
- ✓ Asigna permisos por esquema.

BENEFICIOS DE USAR ESQUEMAS



Mayor seguridad

Permite controlar permisos por esquema.



Mejor organización

Agrupar objetos por módulos o funciones.



Evita conflictos

Permite objetos con el mismo nombre en distintos esquemas.



Mantenimiento más sencillo

Facilita la gestión y actualización.

LISTAR ESQUEMAS EXISTENTES

Consulta los esquemas disponibles en la base de datos:

```
SELECT name
FROM sys.schemas
ORDER BY name;
```

También puedes ver los objetos dentro de un esquema:

```
SELECT *
FROM information_schema.tables
WHERE table_schema = 'Academico';
```

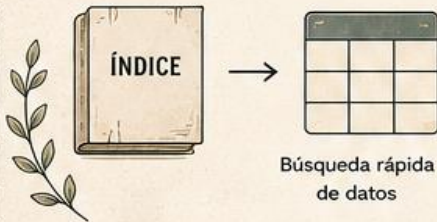
Organiza hoy, facilita el trabajo de mañana.

Los índices mejoran la velocidad de búsqueda y recuperación de datos en las tablas, a cambio de espacio y mantenimiento.



¿QUÉ ES UN ÍNDICE?

Un índice es una estructura que permite localizar rápidamente las filas en una tabla sin necesidad de recorrerla completa.



¿PARA QUÉ SIRVE?

- ✓ Acelera las consultas (SELECT).
- ✓ Mejora el rendimiento en filtros, ordenamientos y joins.
- ✓ Garantiza la unicidad de datos (índices únicos).
- ✓ Reduce el uso de recursos del sistema.
- ✓ Puede mejorar el rendimiento de consultas complejas.

TIPOS DE ÍNDICES



Clustered Index

Define el orden físico de los datos en la tabla. Solo puede haber uno por tabla.



Nonclustered Index

Crea una estructura separada que apunta a las filas de la tabla.



Unique Index

Asegura que los valores de la columna sean únicos.



Filtered Index

Índice basado en un subconjunto de filas que cumplen una condición.

SINTAXIS BÁSICA

1. CREAR ÍNDICE (NONCLUSTERED)

```
CREATE INDEX nombre_indice
ON nombre_tabla (columna1 [ASC|DESC],
columna2 [ASC|DESC], ...);
```

Ejemplo:

```
CREATE INDEX IX_Estudiante_Apellido
ON Academico.Estudiante (Apellido);
```

2. CREAR ÍNDICE ÚNICO

```
CREATE UNIQUE INDEX nombre_indice
ON nombre_tabla (columna);
```

Ejemplo:

```
CREATE UNIQUE INDEX UX_Estudiante_DNI
ON Academico.Estudiante (DNI);
```

3. CREAR ÍNDICE CLUSTERED

```
CREATE CLUSTERED INDEX nombre_indice
ON nombre_tabla (columna);
```

Ejemplo:

```
CREATE CLUSTERED INDEX CX_Estudiante_Id
ON Academico.Estudiante (IdEstudiante);
```

COLUMNAS EN UN ÍNDICE

ASC / DESC

Define el orden de los valores almacenados en el índice.

Ejemplo:

```
CREATE INDEX IX_Nota_Fecha
ON Evaluacion.Nota
(FechaEvaluacion DESC);
```

INCLUIR COLUMNAS

Permite agregar columnas que no son clave para cubrir consultas y evitar accesos a la tabla.

Ejemplo:

```
CREATE INDEX IX_Notas_Estudiante
ON Evaluacion.Nota (IdEstudiante)
INCLUDE (Nota, FechaEvaluacion);
```

VER ÍNDICES DE UNA TABLA

Consultas del sistema para visualizar índices existentes.

```
-- Usando sp_helpindex
EXEC sp_helpindex 'Academico.Estudiante';

-- Vista del sistema
SELECT *
FROM sys.indexes
WHERE object_id = OBJECT_ID('Academico.Estudiante');
```



CONSIDERACIONES IMPORTANTES

- ★ Los índices mejoran las lecturas, pero pueden ralentizar las inserciones, actualizaciones y eliminaciones.
- ★ No todas las columnas deben tener índice.
- ★ Evita crear demasiados índices innecesarios.
- ★ Mantén estadísticas actualizadas para un óptimo rendimiento.



MANTENIMIENTO DE ÍNDICES

RECONSTRUIR ÍNDICE

Reorganiza el índice eliminando la fragmentación.

```
ALTER INDEX nombre_indice
ON nombre_tabla
REBUILD;
```

REORGANIZAR ÍNDICE

Optimiza el índice sin reconstruirlo completamente.

```
ALTER INDEX nombre_indice
ON nombre_tabla
REORGANIZE;
```

EJEMPLO PRÁCTICO

- 1 Supongamos la tabla Academico.Estudiante con muchos registros.
- 2 Creamos un índice para acelerar búsquedas por Apellido y Nombre.
- 3 Consultamos los datos usando el índice.

```
-- Crear indice
CREATE INDEX IX_Estudiante_Apellido_Nombre
ON Academico.Estudiante (Apellido, Nombre);

-- Consulta que se beneficia del indice
SELECT IdEstudiante, Nombre, Apellido
FROM Academico.Estudiante
WHERE Apellido = 'Fernandez';
```

BUENAS PRÁCTICAS

- ★ Crea índices en columnas usadas en WHERE, JOIN, ORDER BY y GROUP BY.
- ★ Usa índices únicos para garantizar integridad.
- ★ Revisa y elimina índices que no se usan.
- ★ Monitorea el uso de los índices regularmente.



STORE PROCEDURES

EN SQL SERVER

Los store procedures (procedimientos almacenados) son conjuntos de instrucciones SQL precompiladas que se almacenan en la base de datos y pueden reutilizarse.



¿QUÉ ES UN STORE PROCEDURE?

Es un objeto de la base de datos que contiene un conjunto de sentencias SQL para realizar una tarea específica.



Se ejecuta cuando es llamado, mejorando el rendimiento, seguridad y mantenimiento.

¿PARA QUÉ SIRVEN?

- ✓ Encapsulan lógica de negocio.
- ✓ Mejoran el rendimiento (se compilan una sola vez).
- ✓ Aumentan la seguridad (control de permisos).
- ✓ Facilitan el mantenimiento del código.
- ✓ Permiten reutilización de código.
- ✓ Reducen el tráfico en la red.

VENTAJAS

- Seguridad**
Controla quién puede ejecutar y qué datos puede ver.
- Rendimiento**
Código precompilado y ejecución más rápida.
- Mantenimiento**
Los cambios se realizan en un solo lugar.
- Reutilización**
El mismo procedimiento puede ser usado en diferentes aplicaciones.

SINTAXIS BÁSICA

```

CREATE PROCEDURE nombre_procedimiento
    @parametro1 tipo_dato = valor_default, -- opcional
    @parametro2 tipo_dato OUTPUT          -- opcional
AS
BEGIN
    -- Instrucciones SQL
END
GO
  
```

Los parámetros son opcionales pero muy útiles.

ELEMENTOS

- CREATE PROCEDURE: crea el procedimiento.
- @parametro: recibe valores de entrada o salida.
- AS BEGIN END: bloque de instrucciones.
- GO: separador de lotes.

TIPOS DE PARÁMETROS

- IN (por defecto): entrada de datos.
- OUT: devuelve un valor.
- INOUT: entrada y salida.

EJEMPLO: CREAR STORE PROCEDURE

```

-- Procedimiento que obtiene estudiantes por facultad
CREATE PROCEDURE sp_ObtenerEstudiantesPorFacultad
    @IdFacultad INT
AS
BEGIN
    SELECT e.IdEstudiante,
           e.Nombres,
           e.Apellidos,
           f.NombreFacultad
    FROM Academico.Estudiante e
    INNER JOIN Academico.Facultad f
        ON e.IdFacultad = f.IdFacultad
    WHERE e.IdFacultad = @IdFacultad
    AND e.Activo = 1
END
GO
  
```

Este procedimiento recibe el Id de la facultad y devuelve la lista de estudiantes activos.

EJECUTAR STORE PROCEDURE

1. SIN PARÁMETROS

```

-- Procedimiento sin parámetros
EXEC sp_ListarFacultades;
  
```



2. CON PARÁMETROS DE ENTRADA

```

-- Con un parámetro
EXEC sp_ObtenerEstudiantesPorFacultad @IdFacultad = 2;

-- Con varios parámetros
EXEC sp_BuscarEstudiante
    @Nombre = 'Ana',
    @Activo = 1;
  
```



3. CON PARÁMETROS DE SALIDA

```

-- Con parámetro de salida
DECLARE @Total INT;
EXEC sp_ContarEstudiantes @Total OUTPUT;
SELECT @Total AS TotalEstudiantes;
  
```



EJEMPLO: PROCEDIMIENTO CON PARÁMETROS DE SALIDA

```

-- Devuelve el total de estudiantes por facultad
CREATE PROCEDURE sp_ContarEstudiantes
    @IdFacultad INT,
    @TotalEstudiantes INT OUTPUT
AS
BEGIN
    SELECT @TotalEstudiantes = COUNT(*)
    FROM Academico.Estudiante
    WHERE IdFacultad = @IdFacultad AND Activo = 1;
END
GO
  
```

Devuelve el número de estudiantes en @TotalEstudiantes.



MODIFICAR Y ELIMINAR

• Modificar un procedimiento

```

ALTER PROCEDURE sp_ObtenerEstudiantesPorFacultad
    @IdFacultad INT
AS
BEGIN
    -- Nuevas instrucciones
END
GO
  
```



• Eliminar un procedimiento

```

DROP PROCEDURE sp_ObtenerEstudiantesPorFacultad;
GO
  
```



BUENAS PRÁCTICAS

- ★ Usa nombres descriptivos y consistentes.
- ★ Documenta el propósito, parámetros y ejemplo de uso.
- ★ Valida los parámetros de entrada.
- ★ Maneja errores con TRY...CATCH.
- ★ Evita lógica innecesaria dentro del procedimiento.
- ★ Otorga permisos mínimos necesarios.

LOS STORE PROCEDURES HACEN TU CÓDIGO MÁS RÁPIDO, SEGURO, REUTILIZABLE Y FÁCIL DE MANTENER.